# MITRE ATTACK RULE-BASED NAVIGATOR

## A PROJECT REPORT

Submitted by

**SIVARANJANII. M (2116241901109)**

in partial fulfillment for the award of the degree of

**BACHELOR OF ENGINEERING**

in

**COMPUTER SCIENCE AND ENGINEERING**
**(CYBER SECURITY)**




**RAJALAKSHMI ENGINEERING COLLEGE**

An Autonomous Institute

**CHENNAI**

**NOVEMBER 2026**

# RAJALAKSHMI ENGINEERING COLLEGE
# CHENNAI

## BONAFIDE CERTIFICATE

Certified that this project report "**MITRE ATTACK RULE-BASED NAVIGATOR**" is the bonafide work of **SIVARANJANII. M** (241901109) who carried out the project work under my supervision.

**SIGNATURE**

**Dr. S. Bhuvana S, Ph.D,**
**ASSISTANT PROFESSOR (SG)**
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

**SIGNATURE OF THE INTERNAL EXAMINER**

# ABSTRACT

Security analysts spend a great deal of time matching system events, such as PowerShell execution, registry changes or repeated failed logins, to attack techniques in the MITRE ATTACK framework. This work is slow, needs expert knowledge and rarely comes with clear steps to fix the problem. Machine-learning detectors can automate part of it, but they cannot explain their decisions.

This project presents the MITRE ATTACK Rule-Based Navigator, an expert system that uses classical Artificial Intelligence only. A goal-based intelligent agent controls the reasoning. ATTACK knowledge is stored as IF-THEN rules, and forward chaining applies these rules to the observed behaviours to find matching techniques. Best-first search ranks the results using the score matched conditions ÷ total conditions, so the best-supported technique appears first and partly matched rules are shown as possible techniques.

For each technique the system reports its confidence, risk level, recommended mitigations and an explanation of which rules and facts led to the result. The knowledge base holds 18 rules covering six ATTACK tactics. It is written in Python without machine-learning libraries, runs offline on ordinary hardware, and can be used through a command-line menu or a Flask web interface. The project shows how search and inference can support clear, explainable decision-making in security operations.

**Keywords:** MITRE ATTACK, rule-based expert system, forward chaining, best-first search, goal-based agent, explainable artificial intelligence, threat technique mapping, mitigation recommendation, Python, Flask.

# ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for enabling us to complete this project successfully. We express our sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his dedicated efforts in providing quality education at this esteemed institution. We are deeply grateful to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her constant motivation and encouragement, which inspired us throughout the completion of this project. We also thank our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We express our sincere gratitude to our College Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement.

We are especially thankful to our project guide, Dr. Bhuvana S., Ph.D., Assistant Professor (SG), for her valuable guidance, continuous encouragement, and support throughout the project. Finally, we thank our parents and friends for their direct and indirect contributions, encouragement, and support in the successful completion of this project.

**SIVARANJANII. M (2116241901109)**

# TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | ABSTRACT | iii |
| | LIST OF TABLES | vi |
| | LIST OF FIGURES | vii |
| 1 | INTRODUCTION | 1 |
| | 1.1 BACKGROUND | 1 |
| | 1.2 PROBLEM STATEMENT | 1 |
| | 1.3 OBJECTIVES | 2 |
| | 1.4 SCOPE | 2 |
| | 1.5 SECURITY REQUIREMENTS AND CONSIDERATIONS | 2 |
| | 1.6 SDG ALIGNMENT | 3 |
| 2 | LITERATURE REVIEW | 4 |
| | 2.1 EXISTING SYSTEM WITH SECURITY APPROACHES | 4 |
| | 2.2 RESEARCH GAPS | 6 |
| | 2.3 PROPOSED SYSTEM | 7 |
| 3 | SYSTEM DESIGN | 8 |
| | 3.1 SYSTEM ARCHITECTURE | 8 |
| | 3.2 IMPLEMENTATION OF THE NAVIGATOR COMPONENTS | 9 |
| | 3.3 DATA FLOW AND WORKFLOW DIAGRAMS | 11 |
| | 3.4 THREAT MODEL AND ATTACK SURFACE | 13 |
| | 3.5 SECURITY CONTROLS | 14 |
| 4 | SYSTEM DESCRIPTION | 15 |
| | 4.1 DATA COLLECTION | 15 |
| | 4.2 DATA PREPROCESSING | 18 |
| | 4.3 ALGORITHMS AND METHODOLOGY | 19 |
| | 4.4 RESULTS AND DISCUSSION | 21 |
| | 4.5 CHALLENGES AND LIMITATIONS | 25 |
| 5 | CONCLUSION | 26 |
| | 5.1 SUMMARY OF THE WORK | 26 |
| | 5.2 LIMITATIONS | 26 |
| | 5.3 FUTURE WORK | 26 |
| | REFERENCES | 27 |
| | APPENDIX | 29 |

# LIST OF TABLES


| TABLE NO. | TITLE | PAGE NO. |
|---|---|---|
| 1.1 | SDG alignment | 3 |
| 2.1 | Literature survey | 5 |
| 2.2 | What exists and what is missing | 6 |
| 3.1 | Hardware requirements | 9 |
| 3.2 | Software requirements | 9 |
| 3.3 | Threat model of the Navigator | 13 |
| 3.4 | Security controls | 14 |
| 4.1 | Distribution of the knowledge base rules | 15 |
| 4.2 | Sample test scenarios and ranked output | 21 |
| 4.3 | Detection report for the scenario 01_malicious_document | 21 |
| A.1 | Knowledge base rules and their IF conditions | 29 |
| A.2 | Mitigations attached to each rule | 31 |
| B.1 | Key terms | 32 |

# LIST OF FIGURES


| FIGURE NO. | TITLE | PAGE NO. |
|---|---|---|
| 3.1 | System architecture | 8 |
| 3.2 | Level 0 data flow diagram | 11 |
| 3.3 | Level 1 data flow diagram | 12 |
| 3.4 | Methodology and workflow diagram | 13 |
| 4.1 | Knowledge base page: rules R001 to R004 | 16 |
| 4.2 | Knowledge base page: rules R005 to R008 | 17 |
| 4.3 | Knowledge base page: rules R009 to R012 | 17 |
| 4.4 | Knowledge base page: rules R013 to R016 | 18 |
| 4.5 | Knowledge base page: rules R017 and R018 | 18 |
| 4.6 | Recognise–act cycle of the inference engine | 19 |
| 4.7 | Web interface: home page and observation input form | 22 |
| 4.8 | Web interface: alert and detection report | 23 |
| 4.9 | Web interface: recommended mitigations | 24 |
| 4.10 | Web interface: explanation of the reasoning | 24 |

# CHAPTER 1
# INTRODUCTION

## 1.1 BACKGROUND

Every computer produces events all the time: programs start, registry settings change and users try to log in. A security analyst in a Security Operations Centre (SOC) reads these events and decides whether an attack is happening and what kind of attack it is.

### 1.1.1 MITRE ATTACK framework

To describe attacks in a common language, security teams use MITRE ATTACK [1], [12]. It is a public catalogue of attacker behaviour built from real incidents. It rests on two main ideas. A tactic is the attacker's goal at one stage of an attack, for example Credential Access, which means stealing passwords. A technique is the way the goal is reached, for example Brute Force (T1110), which means guessing passwords again and again. ATTACK also lists mitigations for each technique, which are the steps a defender can take to stop or reduce it [13], and the ATTACK Navigator tool displays techniques on the matrix [15].

### 1.1.2 Classical artificial intelligence techniques

This project uses four ideas from classical Artificial Intelligence, as taught in the course CS23533 Foundations of Artificial Intelligence. IF-THEN rules write knowledge in the form "IF these things are seen, THEN this attack is likely". Forward chaining starts from what is observed and keeps applying rules to reach conclusions. Best-first search always examines the most promising option first, using a score called a heuristic [4]. A goal-based agent is a program that observes, reasons and then acts to reach a goal.

The MITRE ATTACK Rule-Based Navigator combines these ideas. It takes observed behaviours and reports ranked ATTACK techniques, a risk level, mitigations and the reasoning behind each answer.

## 1.2 PROBLEM STATEMENT

Today, analysts match raw events, such as PowerShell running, registry changes or many failed logins, to ATTACK techniques by hand. This causes three problems. The work is slow and needs an expert, alerts rarely come with clear steps to fix the problem, and small security teams are hit hardest because they have fewer experts.

Existing tools also fall short. Signature-based tools miss attacks that are recognised by behaviour, and machine-learning tools work like a black box: they give an answer but cannot explain it [8], [10]. A simple, light and explainable tool is therefore needed.

## 1.3 OBJECTIVES

- Build a goal-based agent that maps observed behaviours to ATTACK techniques.
- Store ATTACK knowledge as IF-THEN rules (rules.json) and a technique-to-mitigation table (mitigations.json).
- Use forward chaining to reach conclusions and keep an explanation of every step.
- Use best-first search with the score h(rule) = matched conditions ÷ total conditions to rank the results from 0.0 to 1.0.
- Show the risk level and mitigations through a command-line menu and a Flask web page.
- Test the system on representative attack scenarios and report the results.

## 1.4 SCOPE

### 1.4.1 Coverage of the project

The project delivers a rule-based system written in Python using classical AI only. Input is given as facts written as attribute:value lines, through a CLI menu, a Flask web form, prepared sample scenarios or uploaded fact and log files. The knowledge base holds 18 IF–THEN rules for ATTACK techniques and sub-techniques from six tactics: Initial Access, Execution, Persistence, Credential Access, Defense Evasion and Discovery. The output contains the technique, confidence score, risk level, mitigations and an explanation.

### 1.4.2 Exclusions

The project does not use machine learning or deep learning, does not read raw system logs in real time and does not cover the whole ATTACK matrix. It also does not take action automatically: the system only recommends mitigations.

## 1.5 SECURITY REQUIREMENTS AND CONSIDERATIONS

Because this is a security tool, it must itself be trustworthy, and six requirements were therefore kept in mind. Broken or duplicate rules must be rejected so that wrong conclusions are not produced. Observations must follow the attribute:value format so that unexpected input cannot disturb the reasoning. Every answer must show which rules and facts led to it, and the reasoning must always stop, even when rules affect each other. The tool must work

offline so that incident details stay on the analyst's machine, and confidence must come from a simple, documented formula. Sections 3.4 and 3.5 examine the threats to the system and the controls that address them.

## 1.6 SDG ALIGNMENT

Table 1.1 links the project to the United Nations Sustainable Development Goals (SDGs) [20].

**Table 1.1 SDG alignment**

| S. No. | SDG No. and goal | Relevant SDG target | Project contribution / alignment |
|---|---|---|---|
| 1 | SDG 9 – Industry, Innovation and Infrastructure | 9.1 – Develop quality, reliable, resilient infrastructure | Helps small security teams spot attacks on their systems using a light tool that needs no special hardware. |
| 2 | SDG 9 – Industry, Innovation and Infrastructure | 9.5 – Enhance scientific research and upgrade technological capabilities | Shows that classical AI (search and inference) can solve a real security problem without large datasets. |
| 3 | SDG 4 – Quality Education | 4.4 – Increase the number of people with relevant technical skills | Every decision can be traced to a rule, so the tool also helps students learn how ATTACK and AI reasoning work. |

# CHAPTER 2
# LITERATURE REVIEW

## 2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

### 2.1.1 Literature survey

Eleven works form the base of this project, and they are numbered in Table 2.1 in the same order as in the References chapter. Strom et al. [1] created MITRE ATTACK, a knowledge base of attacker tactics and techniques built from real-world behaviour. It gives the whole security community one shared vocabulary, but it is a reference catalogue only and does not tell an analyst which technique is happening in live events.

Lindqvist and Porras [2] built P-BEST, an expert system for detecting computer misuse with production rules and forward chaining. It shows that rule-based reasoning works for intrusion detection, but it was published in 1999, before ATTACK existed, so its rules are not tied to a standard framework, and it gives no mitigation advice and does not rank its rules. Husari et al. [3] built TTPDrill, which reads written threat reports and maps the text to ATTACK techniques using an ontology and language processing. This works on text written after an incident, not on behaviour observed as it happens.

Hart, Nilsson and Raphael [4] described heuristic best-first search, the A* family, in which a priority queue always picks the most promising option first. This is the idea used here to rank techniques, but the paper is general, so a suitable score for security hypotheses had to be designed. Forgy [5] introduced the Rete algorithm, a fast way to match many facts against many rules. It speeds up matching but does not rank partial matches and contains no security knowledge.

Denning [6] proposed an intrusion-detection model that checks audit records against rules and profiles of normal behaviour, and Ilgun, Kemmerer and Porras [7] represented attacks as state transitions in the STAT approach. Both are early rule-based detectors that predate ATTACK and give no technique mapping or mitigation guidance. Sommer and Paxson [8] explain why machine-learning detection is hard to use in real networks, including the difficulty of explaining its alerts. Kwon et al. [9] map ATTACK techniques to NIST Cybersecurity Framework controls, but the mapping is static and does not reason over observed behaviour. Adadi and Berrada [10] and Capuano et al. [11] survey explainable AI in general and in cybersecurity, and both stress that security decisions should be explainable, which supports the rule-based design chosen here.

**Table 2.1 Literature survey**

| S. No. | Author(s) and year | Paper title | Methodology / algorithm | Limitations |
|---|---|---|---|---|
| 1 | B. E. Strom et al. (2018) [1] | MITRE ATT&CK: Design and Philosophy | Knowledge base of adversary tactics and techniques built from observed real-world behaviour | A reference catalogue only; no runtime reasoning that maps live observations to techniques |
| 2 | U. Lindqvist and P. A. Porras (1999) [2] | Detecting Computer and Network Misuse Through the Production-Based Expert System Toolset (P-BEST) | Forward-chaining production rules for misuse detection | Predates ATTACK; rules not tied to a standard framework; no mitigation guidance or rule prioritisation |
| 3 | G. Husari et al. (2017) [3] | TTPDrill: Automatic and Accurate Extraction of Threat Actions from Unstructured Text of CTI Sources | Ontology and NLP-based mapping of threat-report text to ATTACK techniques | Works on written reports after an incident, not on live observed behaviour |
| 4 | P. E. Hart, N. J. Nilsson and B. Raphael (1968) [4] | A Formal Basis for the Heuristic Determination of Minimum Cost Paths | Heuristic best-first search with a priority queue (A*) | General-purpose; needs a domain heuristic to rank security hypotheses |
| 5 | C. L. Forgy (1982) [5] | Rete: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem | Efficient pattern matching for production (rule-based) systems | Speeds up matching only; does not rank partial matches or supply security knowledge |
| 6 | D. E. Denning (1987) [6] | An Intrusion-Detection Model | Rule and profile based detection from audit records | Predates ATTACK; no technique mapping, mitigations or ranking |
| 7 | K. Ilgun, R. A. Kemmerer and P. A. Porras (1995) [7] | State Transition Analysis: A Rule-Based Intrusion Detection Approach | Attack scenarios modelled as state transitions | Scenario-specific rules; no ATTACK link and no ranking of partial matches |
| 8 | R. Sommer and V. Paxson (2010) [8] | Outside the Closed World: On Using Machine Learning for Network Intrusion Detection | Analysis of the limits of machine learning for intrusion detection | Identifies the problem but offers no explainable technique-mapping tool |

| S. No. | Author(s) and year | Paper title | Methodology / algorithm | Limitations |
|---|---|---|---|---|
| 9 | R. Kwon et al. (2020) [9] | Cyber Threat Dictionary Using MITRE ATT&CK Matrix and NIST Cybersecurity Framework Mapping | Static mapping of ATTACK techniques to NIST framework controls | No inference over observed behaviour; no ranking of evidence |
| 10 | A. Adadi and M. Berrada (2018) [10] | Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI) | Survey of explainable AI methods and the need for explanation | General AI survey; contains no security rules or ATTACK knowledge |
| 11 | N. Capuano et al. (2022) [11] | Explainable Artificial Intelligence in CyberSecurity: A Survey | Survey of explainable AI applied to cybersecurity tasks | A survey only; no implemented ATTACK-based reasoning tool |

### 2.1.2 Tools used in practice today

Signature-based intrusion detection systems and antivirus tools detect only known signatures and miss behaviour-based attacks. Manual mapping by analysts is slow and depends on experts. Machine-learning detectors are black boxes that cannot explain their decisions, and many alerts arrive without mitigations, so they rarely say what to do next. Plain rule scanning checks rules one by one without putting the best-supported rule first, and commercial tools are heavy and hide their reasoning from the user.

## 2.2 RESEARCH GAPS

Putting the survey together shows what exists and what is missing, as summarised in Table 2.2.

**Table 2.2 What exists and what is missing**

| Existing work | What it gives | What is missing |
|---|---|---|
| ATTACK [1] | Standard list of techniques | No reasoning on live events |
| P-BEST [2], rule-based detectors [6], [7] | Forward-chaining or state-based rules | No ATTACK link, no mitigations, no ranking |
| TTPDrill [3] | Text reports mapped to ATTACK | Works after the incident, on text only |
| ATTACK to NIST mapping [9] | Static technique-to-control dictionary | No reasoning on observed behaviour |

| Existing work | What it gives | What is missing |
|---|---|---|
| A* search [4], Rete [5] | Ranking and fast matching | No security knowledge, no ranking of partial matches |
| Machine learning [8]; explainable AI surveys [10], [11] | Automatic detection; need for explanation | Black-box output; no explainable mapping to ATTACK |

The research gap is therefore that there is no light, explainable system that maps observed behaviour directly to ATTACK, ranks the possible techniques by how strong the evidence is, and recommends mitigations.

## 2.3 PROPOSED SYSTEM

### 2.3.1 Overview of the proposed system

The proposed system is the MITRE ATTACK Rule-Based Navigator, a Python expert system that works in four steps. The agent first collects the observed behaviours. Best-first search then scores each rule by how many of its conditions are matched and looks at the best-supported rule first. Forward chaining fires the rules whose conditions are all satisfied and records the conclusions, and finally the system reports the technique, risk level, mitigations and the reasoning.

### 2.3.2 Objectives of the proposed system

The system maps observed behaviour to ATTACK techniques while it runs and ranks techniques by confidence instead of giving a flat list. It shows partly matched rules as possible techniques instead of dropping them, gives a risk level and mitigations for each technique, and explains every conclusion.

### 2.3.3 Advantages of the proposed system

The system works on observed behaviour, while existing work maps written reports after an incident [3]. It is explainable, because every decision is traceable to an IF–THEN rule, unlike a black-box model [10]. The best-supported rule is checked first instead of scanning rules one by one. It is light, using Python only with no training data and no machine-learning libraries, and it works offline, so observations never leave the analyst’s machine.

# CHAPTER 3
# SYSTEM DESIGN

## 3.1 SYSTEM ARCHITECTURE

### 3.1.1 Architecture and components

Figure 3.1 shows the architecture. The agent takes observations from the input layer, reasons with best-first search and forward chaining, and gives the result through the output layer.

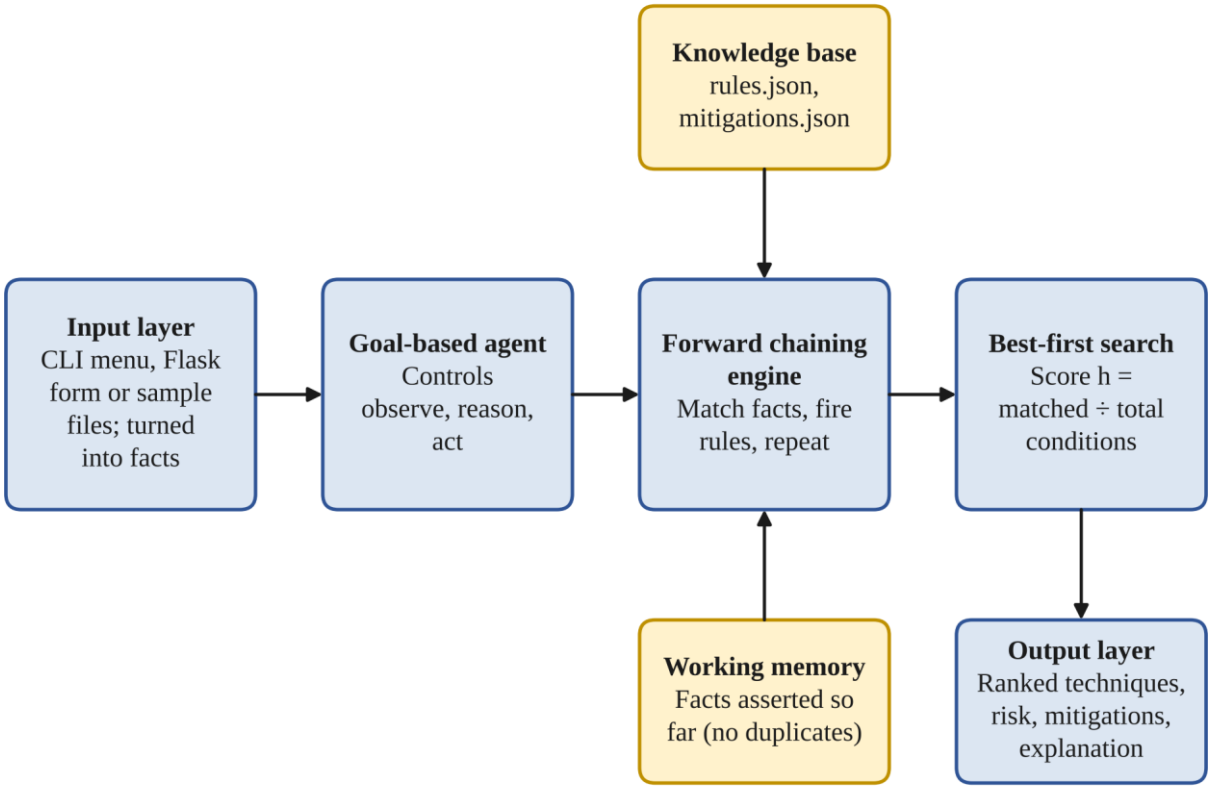



**Fig. 3.1 System architecture**

The input layer takes observations from the CLI menu, the Flask form, prepared sample scenarios or uploaded files and turns them into facts. The goal-based agent controls the observe, reason and act steps and connects all parts. Best-first search gives each rule a score (matched ÷ total conditions) and picks the best first. The forward chaining engine matches facts with rules, fires eligible rules and adds conclusions until nothing more can fire. The knowledge base holds rules.json, which contains the rules, and mitigations.json, which contains the mitigations for each technique. Working memory stores the facts known so far without duplicates, and the output layer shows ranked techniques, risk, mitigations and the explanation.

### 3.1.2 Hardware and software requirements

No GPU and no machine-learning libraries are needed. The requirements are listed in Table 3.1 and Table 3.2. The Python modules and the Flask framework are described in the official documentation [16], [18].

**Table 3.1 Hardware requirements**

| Component | Requirement |
|---|---|
| Processor | Intel Core i3 or higher |
| RAM | 4 GB minimum (8 GB recommended) |
| Storage | Approximately 2 GB free disk space |
| GPU | Not required |

**Table 3.2 Software requirements**

| Component | Requirement |
|---|---|
| Operating system | Windows 10, Linux or macOS |
| Language | Python 3.10 or newer |
| Standard library | heapq, json, dataclasses, collections |
| Web interface | Flask |
| Console reports | tabulate, colorama |
| Optional | matplotlib and pandas (charts and CSV logs); Jupyter (demonstrations) |
| Tools | VS Code, Git and GitHub, virtual environment (venv) |
| Knowledge source | MITRE ATTACK Enterprise matrix (no dataset or training data) |

## 3.2 IMPLEMENTATION OF THE NAVIGATOR COMPONENTS

The Navigator is implemented as five cooperating components, described in Sections 3.2.1 to 3.2.5. Together they form the complete working system.

### 3.2.1 Knowledge base and facts

This component defines how knowledge is stored. A fact is one observed behaviour written as attribute:value, for example process_created:powershell.exe or failed_login_count_exceeds:5. A rule has a rule ID, a list of IF conditions, THEN conclusions, a tactic and a risk level. The KnowledgeBase class loads the 18 rules from rules.json and rejects broken rules and duplicate IDs, and mitigations.json links each

technique to its mitigations. When a rule fires it asserts two derived facts, technique_detected and tactic_detected, which later rules can use as conditions. Rule R007 is shown below as the knowledge base page displays it, and the complete rule set is listed in Appendix A.

```
R007  Medium  Credential Access
IF   failed_login_count_exceeds:5
     same_source_ip:true
     time_window_minutes:10
THEN technique_detected:T1110.001
     tactic_detected:Credential Access
```

### 3.2.2 Forward chaining engine

Working memory stores the facts and avoids duplicates. The engine repeats a cycle of matching the rules, collecting the eligible rules, firing one and adding its conclusion. It also counts how many conditions of each rule are already satisfied, which is called a partial match, and the best-first search component uses this count. An explanation trace records which rule fired, on which facts, and in what order. Because the conclusions of a fired rule are added to working memory as new facts, one rule can enable another, which is called rule chaining. The engine stops when no rule can fire, a state called quiescence or the fixed point.

### 3.2.3 Best-first search

The score of a rule is h(rule) = matched conditions ÷ total conditions, a value from 0.0 to 1.0. A priority queue (heapq) always returns the best-scored rule first [18]. If two rules have the same score, the one with the higher-severity tactic goes first. Rules that are only partly matched are reported as possible techniques.

### 3.2.4 Agent, input layer and report

The Agent class connects everything: it observes, reasons with the engine and the search, and then acts by recommending mitigations. The input layer reads facts from a CLI menu, from typed attribute:value lines, from prepared sample scenarios and from uploaded .facts, .txt and .log files. In a .log file each line is one event, with a timestamp followed by key=value fields, and lines starting with # are ignored. The report shows the techniques, confidence, evidence and mitigations, and the system was tested on prepared attack scenarios, which are reported in Section 4.4.

### 3.2.5 Flask web interface

The Flask web interface sits on top of the same agent and engine [16]. Its Analyse page offers three ways to give evidence: choosing one of the prepared sample scenarios, typing facts, or uploading a file. Its Knowledge base page lists every rule with its conditions, conclusions, risk level and mitigations. The detection report summarises the number of techniques detected, the highest severity, the near misses and the number of inference cycles. It then shows the tactics reached, the detected techniques ranked by confidence and then severity, the detection chain of fired rules, the recommended mitigations, the near misses, the observed facts and a written explanation of the reasoning. A question box also lets the analyst ask why a particular technique was not detected.

## 3.3 DATA FLOW AND WORKFLOW DIAGRAMS

### 3.3.1 Level 0 data flow diagram

Figure 3.2 is the simplest view: the user gives observations to the Navigator and receives a ranked report.

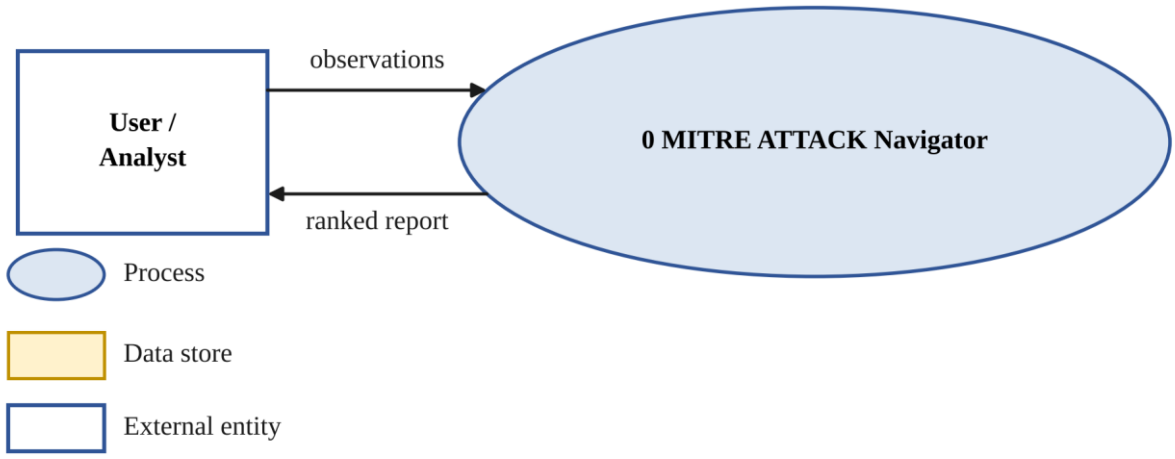



**Fig. 3.2 Level 0 data flow diagram**

### 3.3.2 Level 1 data flow diagram

Figure 3.3 opens the Navigator into four processes. Process 1.0 parses the user's observations into facts and stores them in D2 (working memory). Process 2.0 reads the facts from D2 and the rules from D1 (rules.json) and finds the fully and partly matched rules. Process 3.0 ranks the matches by score using best-first search. Process 4.0 fires the rules, stores the conclusions

in D2, reads the mitigations from D3 (mitigations.json) and sends the report to the user. Because the conclusions return to D2, matching can be repeated until no rule can fire.

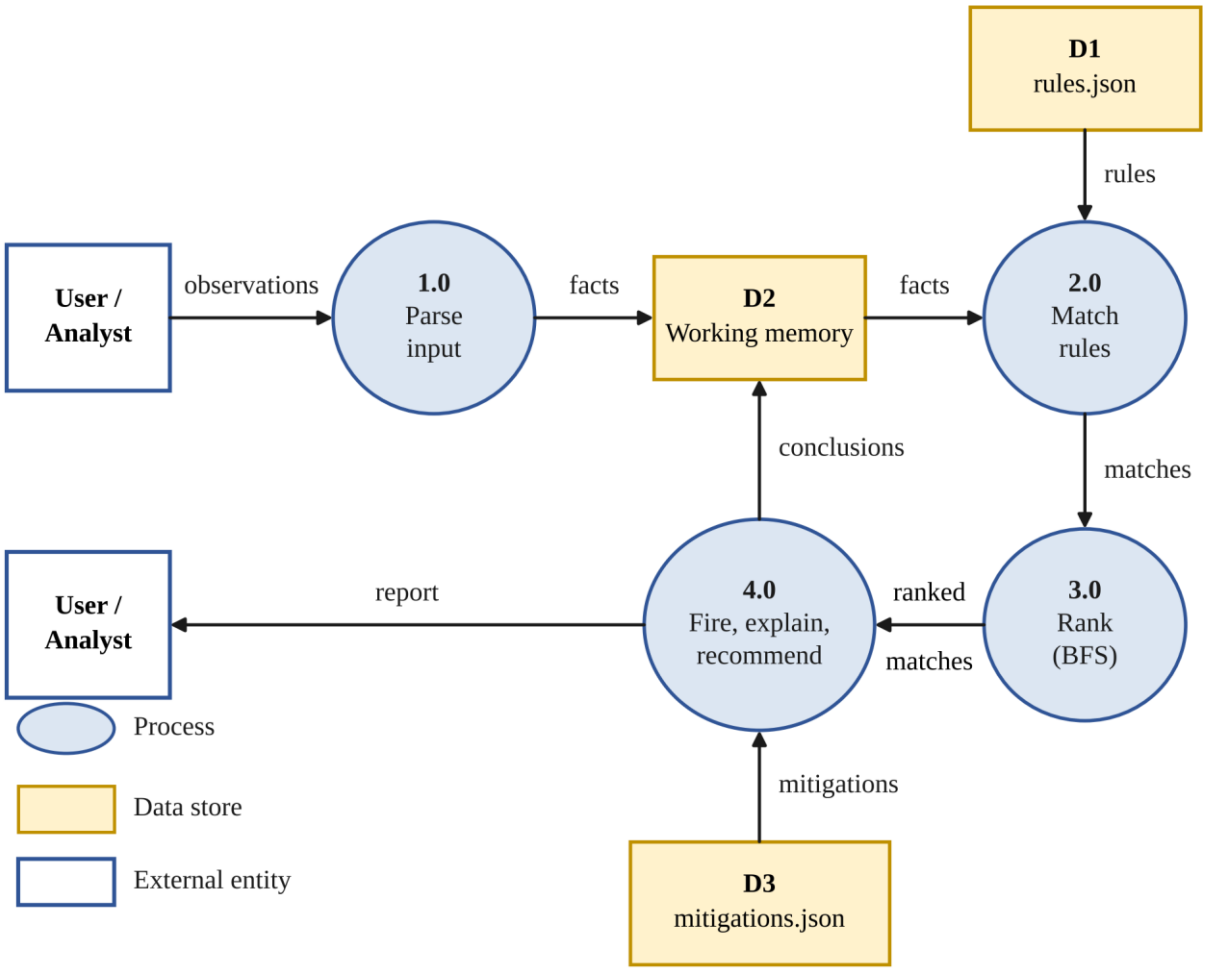



**Fig. 3.3 Level 1 data flow diagram**

### 3.3.3 Methodology and workflow

Figure 3.4 shows the full workflow from observation to report. No data collection or training is needed because only search and inference are used.

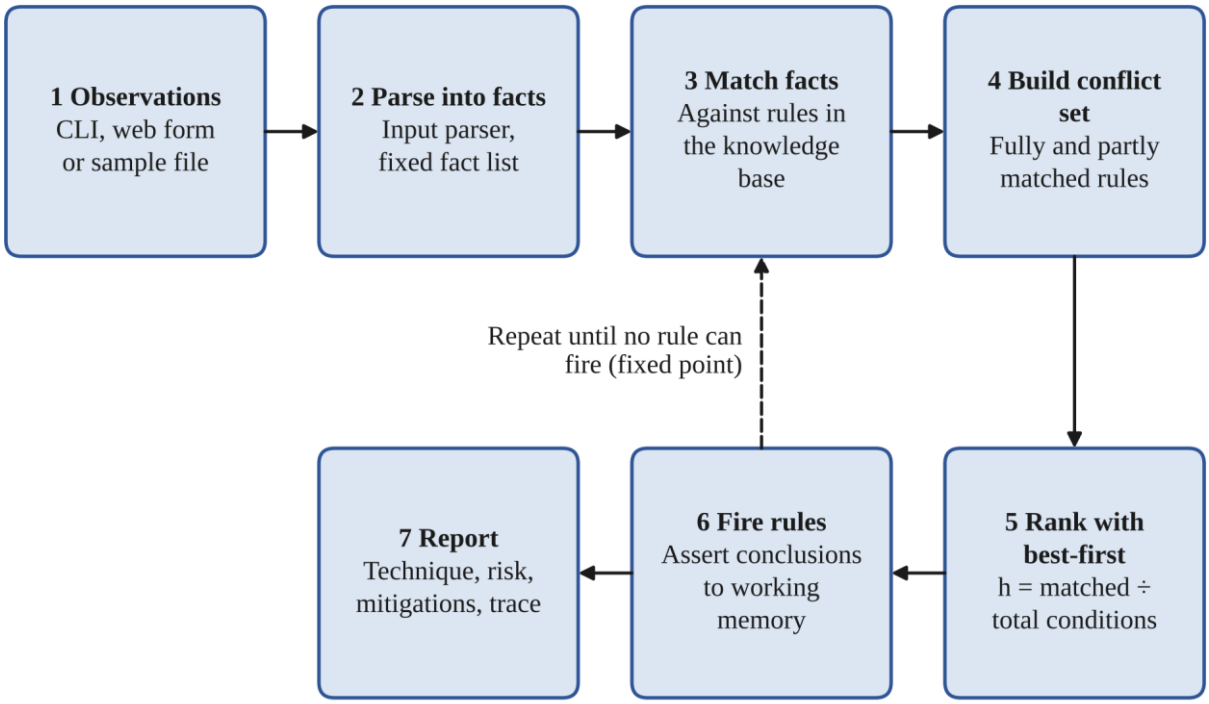



**Fig. 3.4 Methodology and workflow diagram**

## 3.4 THREAT MODEL AND ATTACK SURFACE

Even a small offline tool can be attacked. The assets are the rule files, the observations and the reports, and the entry points are the CLI menu, the Flask form and the sample input files. Table 3.3 lists the main threats, each with an identifier that is used again in Table 3.4. The table is an analysis of the design, not a record of penetration tests [19].

**Table 3.3 Threat model of the Navigator**

| ID | Threat | Asset / entry point | Impact |
|---|---|---|---|
| TH1 | Rule files changed or deleted | rules.json, mitigations.json | Attacks go undetected or the wrong mitigation is shown |
| TH2 | Broken, duplicate or malformed rules | rules.json | Wrong or missing conclusions |
| TH3 | Broken or unexpected input | CLI menu, Flask form, input files | Crash or wrong match |

| ID | Threat | Asset / entry point | Impact |
|---|---|---|---|
| TH4 | Script or markup typed into the web form and shown back | Flask interface | Code runs in the analyst's browser |
| TH5 | Very large or repeated input | All input channels | Analysis machine slows down or hangs |
| TH6 | Reasoning that never stops or repeats rules | Inference engine | Analysis hangs and results are never produced |
| TH7 | Reports seen by others when the web page is reachable | Flask interface, saved reports | Incident details exposed |
| TH8 | Attacker who knows the rules avoids them | Rule coverage | Behaviour not in the rule base is missed |

## 3.5 SECURITY CONTROLS

Table 3.4 gives at least one control for every threat in Table 3.3. Built-in controls are part of the implemented design. Deployment practices are operating measures that the user applies when the tool is run, and they are not part of the program logic.

**Table 3.4 Security controls**

| Threat ID | Control | Type of control |
|---|---|---|
| TH1 | Keep rules.json and mitigations.json read-only and check a stored hash before loading | Deployment practice |
| TH2 | The loader rejects malformed rules and duplicate rule IDs | Built-in control |
| TH3 | Input parser accepts only attribute:value lines and ignores comments; a fact that no rule uses takes no part in the reasoning | Built-in control |
| TH4 | Use Flask template auto-escaping and never place raw input in the page [17] | Built-in framework control and coding practice |
| TH5 | Limit the size and number of observations accepted | Deployment practice |
| TH6 | No duplicate facts in working memory; reasoning stops when no rule can fire | Built-in control |
| TH7 | Works offline; run the web interface on localhost and avoid debug mode on shared networks [17] | Built-in control and deployment practice |
| TH8 | Explanation trace and documented score make gaps visible; extend rules as ATTACK is updated | Built-in control and maintenance practice |

# CHAPTER 4
# SYSTEM DESCRIPTION

## 4.1 DATA COLLECTION

The Navigator has no dataset and no training. Two kinds of information are collected instead.

### 4.1.1 Knowledge

Knowledge is taken from the MITRE ATTACK Enterprise matrix [12], [13] (technique IDs, names, descriptions, tactics and M-code mitigations). It is written as 18 IF-THEN rules in rules.json, with the mitigations in mitigations.json. MITRE publishes the same data as a STIX/JSON bundle [14], which can be used to fill in technique names automatically. The 18 rules cover six tactics, and Table 4.1 shows how they are distributed by tactic and risk level.

**Table 4.1 Distribution of the knowledge base rules**

| Tactic | Rules | Rule IDs | High | Medium | Low |
|---|---|---|---|---|---|
| Initial Access | 1 | R018 | 1 | 0 | 0 |
| Execution | 3 | R001–R003 | 2 | 1 | 0 |
| Persistence | 3 | R004–R006 | 3 | 0 | 0 |
| Defense Evasion | 4 | R010–R013 | 1 | 3 | 0 |
| Credential Access | 3 | R007–R009 | 1 | 2 | 0 |
| Discovery | 4 | R014–R017 | 0 | 0 | 4 |
| **Total** | **18** | R001–R018 | **8** | **6** | **4** |

Each rule lists its IF conditions, the THEN conclusions, a risk level and its mitigations. There are 18 mitigations: 17 are MITRE mitigations identified by M-codes, and one, M9001 Detection-Only Guidance, is team-defined. M9001 is used for the Discovery techniques because ATTACK lists no preventive mitigations for them, and the web interface marks it with an asterisk. Rule R018 shows chaining: its conditions are the derived fact technique_detected:T1059.001 and parent_process:winword.exe, so it can fire only after R001 has fired. Figures 4.1 to 4.5 show the knowledge base page of the web interface, and the full rule listing is given in Appendix A.

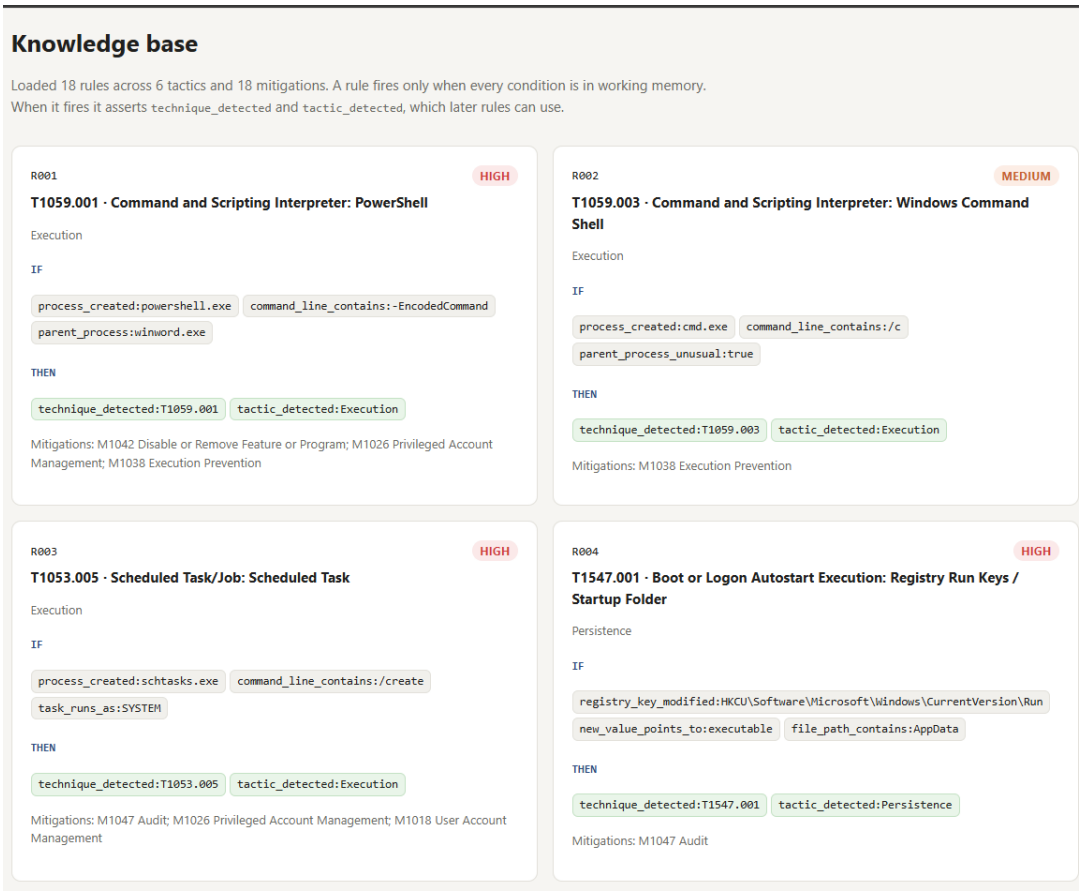



**Fig. 4.1 Knowledge base page: rules R001 to R004**

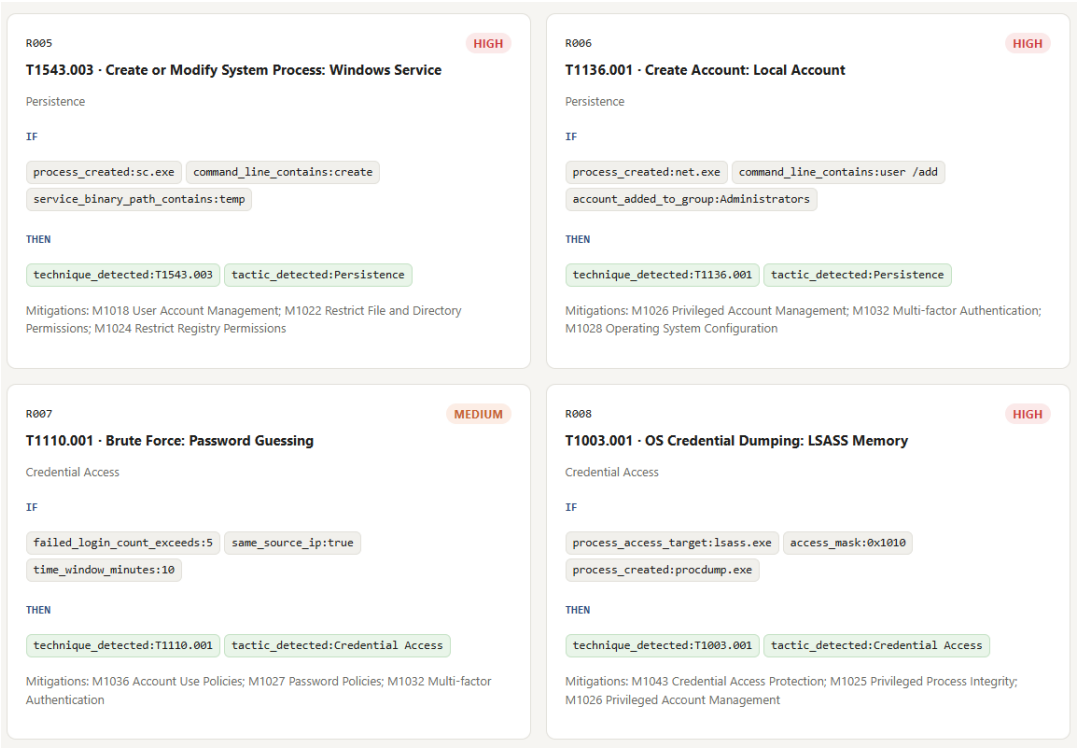



**Fig. 4.2 Knowledge base page: rules R005 to R008**

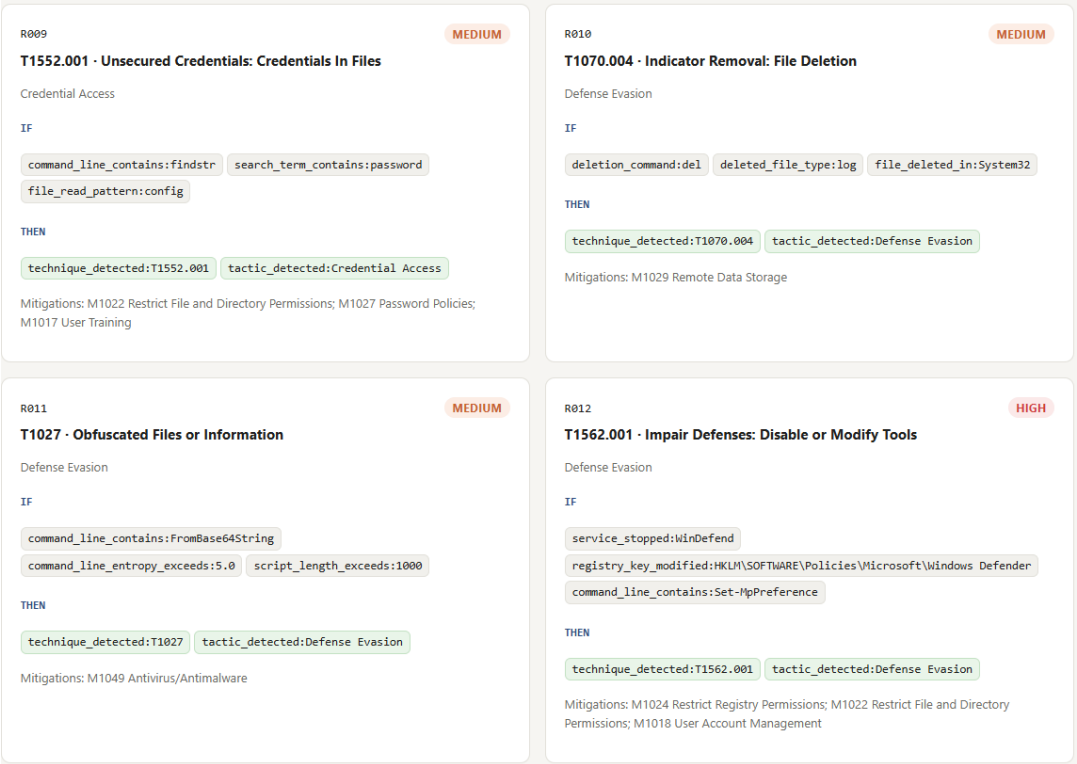



**Fig. 4.3 Knowledge base page: rules R009 to R012**

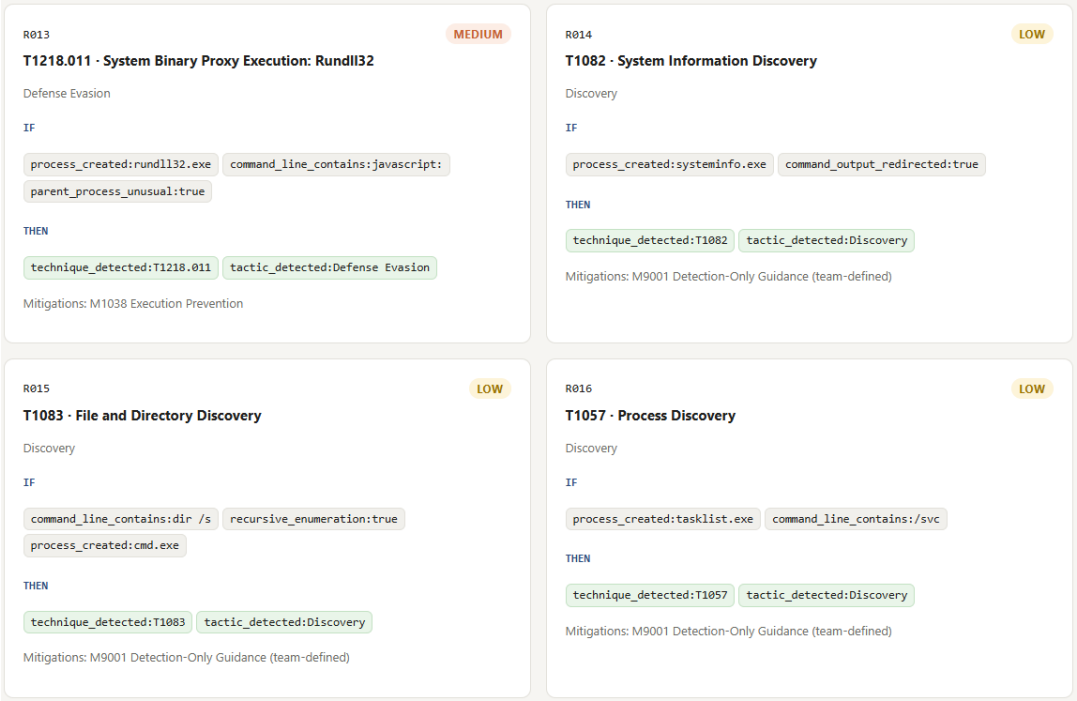



**Fig. 4.4 Knowledge base page: rules R013 to R016**

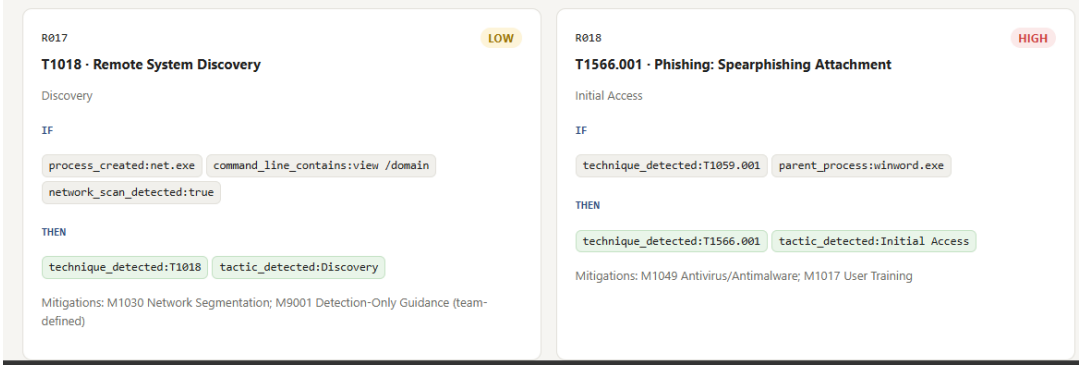



**Fig. 4.5 Knowledge base page: rules R017 and R018**

### 4.1.2 Observations

Observations are the behaviours seen on a system, such as PowerShell running with an encoded command, a changed registry Run key, or more than five failed logins from one source. They are written as attribute:value facts and entered through the CLI menu, the Flask form, the eight prepared sample scenarios or uploaded fact and log files. The prepared scenarios range from a phishing document to normal office activity and include 01_malicious_document, 02_brute_force, 03_persistence and 04_credential_dump.

## 4.2 DATA PREPROCESSING

Real events never match rule text exactly, so input is cleaned before reasoning. The input parser turns each line into a fact written as attribute:value, for example process_created:powershell.exe. In a .log file it first reads the timestamp and then turns each

key=value field of the event into a fact, and it ignores lines that start with # [19]. Each fact is stored once, so repeating an observation does not create duplicates. A fact that no rule uses is still stored, but it is marked as belonging to no rule and takes no part in the reasoning. Raw system logs such as Windows event logs, CSV or JSON are left for future work.

## 4.3 ALGORITHMS AND METHODOLOGY

### 4.3.1 Forward chaining

Forward chaining starts from the known facts and repeats the cycle in Figure 4.6 until no rule can fire. That final state is called a fixed point, or quiescence, and then the report is produced. Each time a rule fires, its conclusions technique_detected and tactic_detected are added to working memory as derived facts, so the next cycle can use them. This is how R018 follows R001 in the worked example of Section 4.3.4.

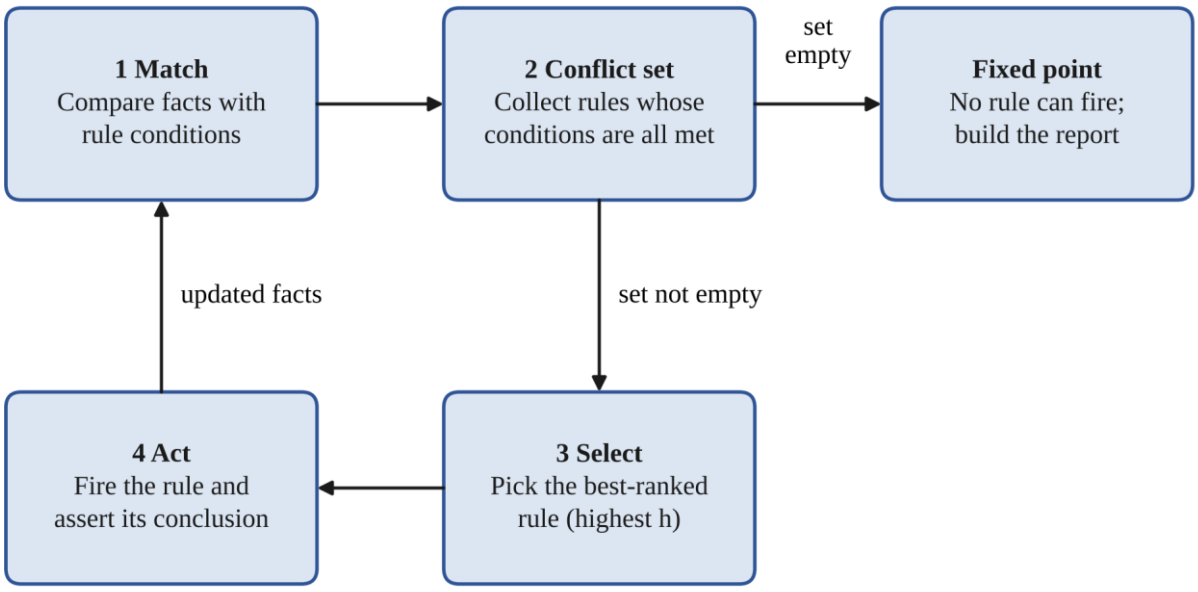



**Fig. 4.6 Recognise–act cycle of the inference engine**

### 4.3.2 Scoring and best-first search

Each rule gets a score h(rule) = matched conditions ÷ total conditions, between 0.0 and 1.0. For example, a rule with three conditions where two are observed scores 2 ÷ 3 = 0.67. A score of 1.0 means the rule is fully supported, so it fires. A score between 0 and 1 means the technique is reported as possible. A priority queue always gives the highest score first, and ties go to the higher-severity tactic. The ranking logic is shown below.

```
for rule in rules:
    h = matched(rule) / total(rule)
    push(queue, (-h, -severity, rule))

while queue is not empty:
    rule = pop(queue)          # best first
    if h == 1.0:
        fire(rule)             # forward chain
        assert conclusion
    elif h > 0:
        report "possible" technique

return techniques ranked by h
```

### 4.3.3 Agent and explanation

The agent observes the input, reasons with the engine and the search, and acts by recommending the mitigations listed for each detected technique. The explanation trace records which rule fired, on which facts and in what order, so every answer can be checked.

### 4.3.4 Worked example

The example below follows the prepared scenario 01_malicious_document, in which a Word document launches encoded PowerShell, which then profiles the host. It shows how three rules fire in three cycles and how R018 uses a fact derived by R001.

```
Cycle 1: R001 fires (h = 1.00)
  IF process_created:powershell.exe
     command_line_contains:-EncodedCommand
     parent_process:winword.exe
  THEN technique_detected:T1059.001
Cycle 2: R018 fires (h = 1.00)
  IF technique_detected:T1059.001 (derived)
     parent_process:winword.exe
  THEN technique_detected:T1566.001
Cycle 3: R014 fires (h = 1.00)
  IF process_created:systeminfo.exe
     command_output_redirected:true
  THEN technique_detected:T1082
Quiescence: no rule can fire, so reasoning stops
```

## 4.4 RESULTS AND DISCUSSION

### 4.4.1 Test scenarios and ranked output

Table 4.2 shows the ranked output for three representative scenarios, and Table 4.3 shows the detection report that the system produced for the scenario 01_malicious_document.

**Table 4.2 Sample test scenarios and ranked output**

| Scenario | Observed behaviour (facts) | Expected technique | Score | Outcome |
|---|---|---|---|---|
| PowerShell-based intrusion | powershell.exe spawned, encoded command | T1059.001 PowerShell | 1.00 | Fires (h = 1.0) |
| Persistence via registry | Run key modified, new startup entry | T1547.001 Registry Run Keys | 0.67 | Possible (near-miss) |
| Credential brute force | More than five failed logins from one source | T1110.001 Password Guessing | 0.50 | Possible (near-miss) |

For 01_malicious_document the report shows 4 lines read and 8 facts. Three techniques were detected, with highest severity High, no near misses and three inference cycles, in which the 8 observed facts were joined by 6 derived facts before the system stopped. The tactics reached were Initial Access, Execution and Discovery.

**Table 4.3 Detection report for the scenario 01_malicious_document**

| # | Technique | Tactic | Severity | Confidence | Rule | Cycle |
|---|---|---|---|---|---|---|
| 1 | T1059.001 PowerShell | Execution | High | 1.00 | R001 | 1 |
| 2 | T1566.001 Spearphishing Attachment | Initial Access | High | 1.00 | R018 | 2 |
| 3 | T1082 System Information Discovery | Discovery | Low | 1.00 | R014 | 3 |

### 4.4.2 Discussion

The results are ranked by confidence, so the best-supported technique comes first. Partly matched rules appear as possible techniques instead of being lost. Compared with checking rules one by one, the score puts the most relevant rule first, and every result has an explanation. The absence of duplicate facts and the fixed-point stop prevent rules from firing again and again.

### 4.4.3 User interface

Figures 4.7 to 4.10 show the web interface of the working system for the scenario 01_malicious_document. Figure 4.7 shows the three ways of giving evidence, Figure 4.8 the detection report, Figure 4.9 the recommended mitigations and Figure 4.10 the written explanation of the reasoning.

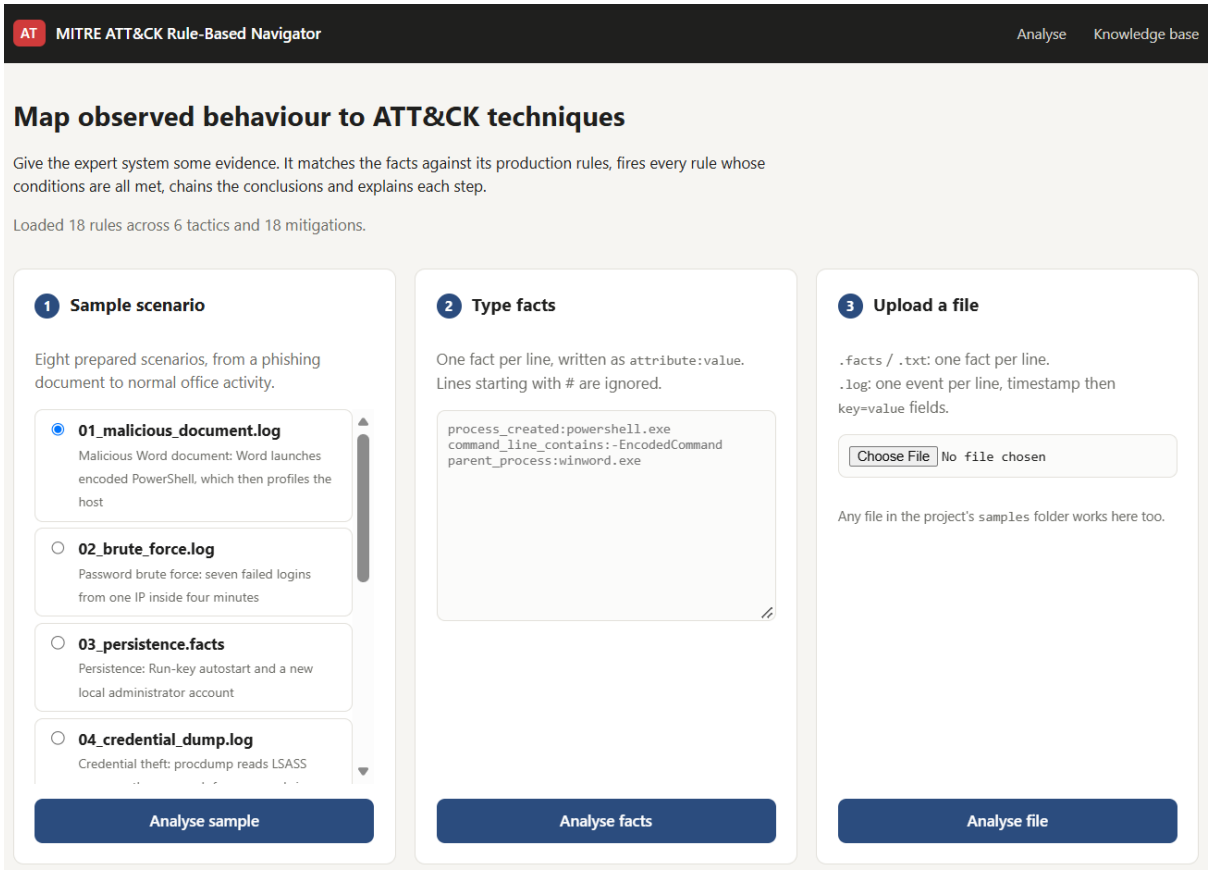



**Fig. 4.7 Web interface: home page and observation input form**

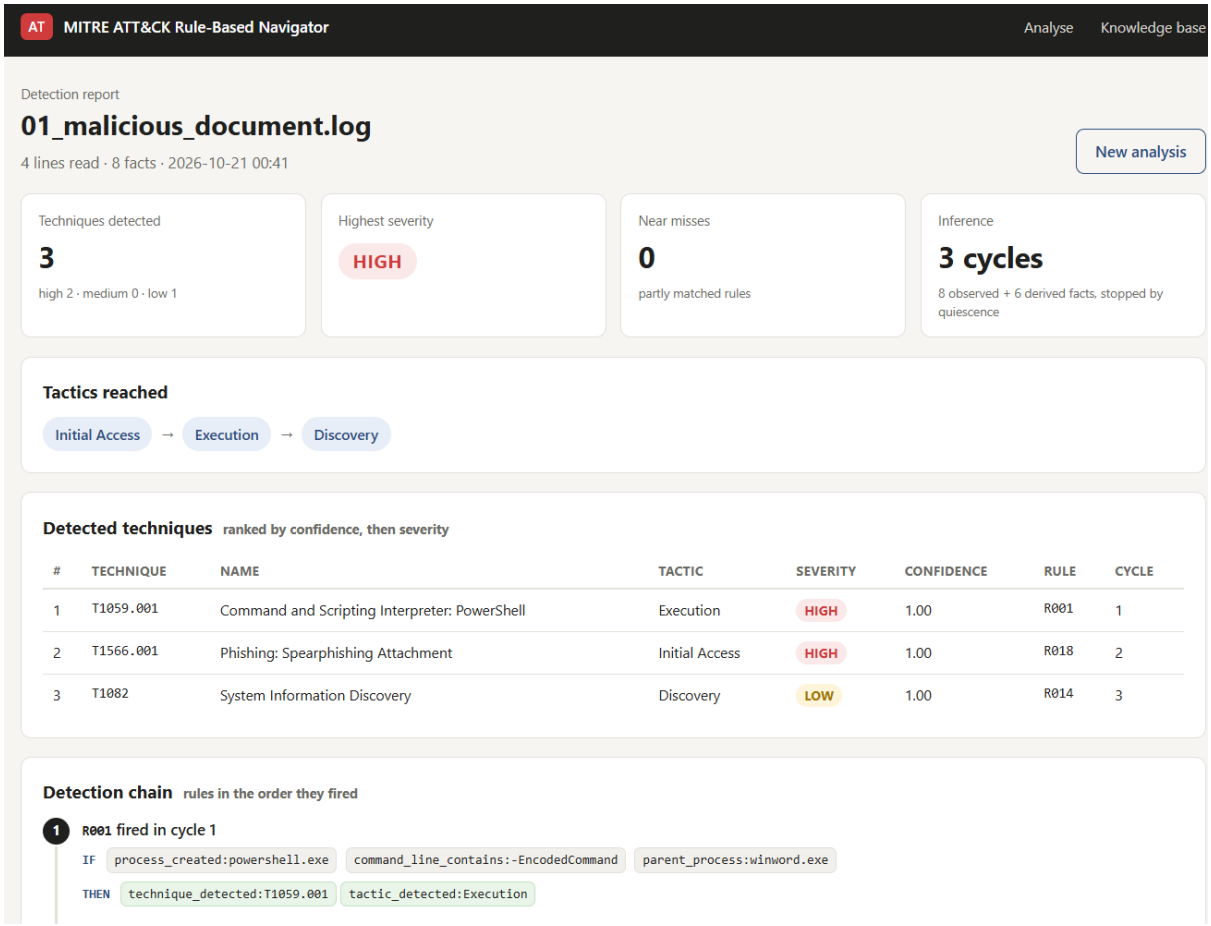



**Fig. 4.8 Web interface: alert and detection report**

**Recommended mitigations**

| ID | MITIGATION | ADDRESSES |
|---|---|---|
| M1042 | Disable or Remove Feature or Program | T1059.001 |
| M1026 | Privileged Account Management | T1059.001 |
| M1038 | Execution Prevention | T1059.001 |
| M1049 | Antivirus/Antimalware | T1566.001 |
| M1017 | User Training | T1566.001 |
| M9001 * | Detection-Only Guidance (team-defined) | T1082 |

* M9001 is team-defined, not an official ATT&CK mitigation. ATT&CK lists no preventive mitigations for Discovery techniques.

**Near misses / possible techniques** some conditions matched, rule did not fire

None.

**Ask: why was a technique not detected?**

T1059.001 - Command and Scripting Interpreter: PowerShell (R001) Explain

**Observed facts** what went into working memory

| # | FACT | FROM | CONDITION OF |
|---|---|---|---|
| 1 | process_created:winword.exe | line 3 | (no rule) |
| 2 | parent_process:explorer.exe | line 3 | (no rule) |

**Fig. 4.9 Web interface: recommended mitigations**

**Why** reasoning behind each detection

```
T1059.001 - Command and Scripting Interpreter: PowerShell
  concluded by rule R001 (cycle 1, tactic: Execution, severity: high)
  because ALL of these facts were in working memory:
    [observed] process_created:powershell.exe
    [observed] command_line_contains:-EncodedCommand
    [observed] parent_process:winword.exe

T1566.001 - Phishing: Spearphishing Attachment
  concluded by rule R018 (cycle 2, tactic: Initial Access, severity: high)
  because ALL of these facts were in working memory:
    [derived by R001] technique_detected:T1059.001
        T1059.001 - Command and Scripting Interpreter: PowerShell
          concluded by rule R001 (cycle 1, tactic: Execution, severity: high)
          because ALL of these facts were in working memory:
            [observed] process_created:powershell.exe
            [observed] command_line_contains:-EncodedCommand
            [observed] parent_process:winword.exe
    [observed] parent_process:winword.exe

T1082 - System Information Discovery
  concluded by rule R014 (cycle 3, tactic: Discovery, severity: low)
  because ALL of these facts were in working memory:
    [observed] process_created:systeminfo.exe
    [observed] command_output_redirected:true
```

**Fig. 4.10 Web interface: explanation of the reasoning**

The web interface titles in these screenshots show the name ATT&CK because they are captured from the running application.

## 4.5 CHALLENGES AND LIMITATIONS

Real events do not match rule text exactly, which was solved with the attribute:value fact format and an input parser. A plain match or no-match answer hid useful evidence, which was solved with the matched ÷ total score and the reporting of possible techniques. Ties and repeated firing were solved with severity tie-breaks, no duplicate facts and a fixed-point stop. The limits that remain are that the rule base covers only part of ATTACK and that the inputs are attribute:value facts and structured log lines, not raw system logs.

# CHAPTER 5
# CONCLUSION

## 5.1 SUMMARY OF THE WORK

This project built a goal-based expert system that maps observed behaviours to MITRE ATTACK techniques using classical AI. IFTHEN rules with forward chaining give clear, traceable decisions. Best-first search with the score matched ÷ total conditions ranks techniques by how strong the evidence is. Every result comes with a risk level, mitigations and an explanation, through a CLI and a Flask web interface.

The project shows that search and inference alone can support practical security reasoning, without machine learning, without training data and on ordinary hardware.

## 5.2 LIMITATIONS

The rule base covers only a subset of ATTACK, the inputs are attribute:value facts and structured log lines, and accuracy has so far been checked on a small set of scenarios.

## 5.3 FUTURE WORK

- Add more ATTACK techniques and tactics to the rule base.
- Accept raw log formats such as Windows event logs, CSV or JSON as input.
- Add more attack scenarios and measure detection accuracy.
- Show the detection chain and confidence scores as charts in the web interface, in addition to the tables.
- Link techniques to detection guidance and keep rules in step with ATTACK updates.

# REFERENCES


[1] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington and C. B. Thomas, “MITRE ATT&CK: Design and Philosophy,” The MITRE Corporation, Tech. Rep., 2018 (revised 2020). **Relevance:** Defines the tactics and techniques model on which every rule in the Navigator is based.

[2] U. Lindqvist and P. A. Porras, “Detecting computer and network misuse through the Production-Based Expert System Toolset (P-BEST),” in Proc. IEEE Symp. Security and Privacy, Oakland, CA, USA, 1999, pp. 146–161. **Relevance:** Shows that forward-chaining production rules work for intrusion detection, which supports the inference design.

[3] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu and X. Niu, “TTPDrill: Automatic and accurate extraction of threat actions from unstructured text of CTI sources,” in Proc. Annual Computer Security Applications Conf. (ACSAC), 2017. **Relevance:** Maps threat-report text to ATTACK techniques and marks the gap that live behaviour mapping fills.

[4] P. E. Hart, N. J. Nilsson and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Trans. Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968. **Relevance:** Source of heuristic best-first search used to rank rules by their score.

[5] C. L. Forgy, “Rete: A fast algorithm for the many pattern/many object pattern match problem,” Artificial Intelligence, vol. 19, no. 1, pp. 17–37, 1982. **Relevance:** Explains efficient matching of facts against rules, the problem solved by the match phase.

[6] D. E. Denning, “An intrusion-detection model,” IEEE Trans. Software Engineering, vol. SE-13, no. 2, pp. 222–232, Feb. 1987. **Relevance:** Foundational rule-based intrusion detection model that motivates rule-driven analysis.

[7] K. Ilgun, R. A. Kemmerer and P. A. Porras, “State transition analysis: A rule-based intrusion detection approach,” IEEE Trans. Software Engineering, vol. 21, no. 3, pp. 181–199, Mar. 1995. **Relevance:** Early rule-based detector that models attacks explicitly, a precursor of ATTACK-based rules.

[8] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in Proc. IEEE Symp. Security and Privacy, Oakland, CA, USA, 2010, pp. 305–316. **Relevance:** Explains why machine learning is hard to trust operationally, which justifies using classical AI.

[9] R. Kwon, Z. Ashley, J. Castleberry, P. Mckenzie and S. N. G. Gourisetti, “Cyber threat dictionary using MITRE ATT&CK matrix and NIST cybersecurity framework mapping,” in Proc. Resilience Week (RWS), IEEE, 2020. **Relevance:** Links ATTACK techniques to defensive controls, similar to the technique-to-mitigation table here.

[10] A. Adadi and M. Berrada, “Peeking inside the black-box: A survey on explainable artificial intelligence (XAI),” IEEE Access, vol. 6, pp. 52138–52160, 2018. **Relevance:** Surveys why AI decisions must be explainable, which supports the explanation trace.

[11] N. Capuano, G. Fenza, V. Loia and C. Stanzione, “Explainable artificial intelligence in cybersecurity: A survey,” IEEE Access, vol. 10, pp. 93575–93600, 2022. **Relevance:** Shows the need for explainable decisions in security tools such as the Navigator.

[12] MITRE, “MITRE ATTACK knowledge base,” https://attack.mitre.org/ (accessed Oct. 2026). **Relevance:** Primary source of the techniques, tactics and descriptions used in the rule base.

[13] MITRE, “Mitigations: Enterprise,” https://attack.mitre.org/mitigations/enterprise/ (accessed Oct. 2026). **Relevance:** Source of the M-code mitigations stored in mitigations.json.

[14] MITRE ATTACK, “attack-stix-data repository,” https://github.com/mitre-attack/attack-stix-data (accessed Oct. 2026). **Relevance:** Machine-readable STIX bundle that can fill in technique names and IDs.

[15] MITRE ATTACK, “ATTACK Navigator,” https://mitre-attack.github.io/attack-navigator/ (accessed Oct. 2026). **Relevance:** Reference tool for viewing techniques on the matrix, which this project complements with reasoning.

[16] Pallets Projects, “Flask documentation,” https://flask.palletsprojects.com/ (accessed Oct. 2026). **Relevance:** Guide used to build the web form and the report pages.

[17] Pallets Projects, “Flask: Security considerations,” https://flask.palletsprojects.com/en/stable/web-security/ (accessed Oct. 2026). **Relevance:** Explains template auto-escaping and safe deployment, which support the web security controls.

[18] Python Software Foundation, “heapq – Heap queue algorithm,” https://docs.python.org/3/library/heapq.html (accessed Oct. 2026). **Relevance:** Documents the priority queue used by best-first search.

[19] OWASP Foundation, “Input validation cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html (accessed Oct. 2026). **Relevance:** Supports strict validation of the attribute:value input format.

[20] United Nations, “The 17 Sustainable Development Goals,” https://sdgs.un.org/goals (accessed Oct. 2026). **Relevance:** Source of the SDG targets used in the alignment table.

# APPENDIX

## A. KNOWLEDGE BASE RULES

This appendix lists all 18 rules of the knowledge base as they appear on the Knowledge base page of the web interface. A rule fires only when every condition in its IF column is in working memory. When it fires it asserts technique_detected and tactic_detected for its technique and tactic.

**Table A.1 Knowledge base rules and their IF conditions**

| Rule | Technique and tactic | IF conditions | Risk |
|---|---|---|---|
| R001 | T1059.001 Command and Scripting Interpreter: PowerShell (Execution) | process_created:powershell.exe<br>command_line_contains:-EncodedCommand<br>parent_process:winword.exe | High |
| R002 | T1059.003 Command and Scripting Interpreter: Windows Command Shell (Execution) | process_created:cmd.exe<br>command_line_contains:/c<br>parent_process_unusual:true | Medium |
| R003 | T1053.005 Scheduled Task/Job: Scheduled Task (Execution) | process_created:schtasks.exe<br>command_line_contains:/create<br>task_runs_as:SYSTEM | High |
| R004 | T1547.001 Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder (Persistence) | registry_key_modified:HKCU\Software\Microsoft\Windows\CurrentVersion\Run<br>new_value_points_to:executable<br>file_path_contains:AppData | High |
| R005 | T1543.003 Create or Modify System Process: Windows Service (Persistence) | process_created:sc.exe<br>command_line_contains:create<br>service_binary_path_contains:temp | High |
| R006 | T1136.001 Create Account: Local Account (Persistence) | process_created:net.exe<br>command_line_contains:user /add<br>account_added_to_group:Administrators | High |
| R007 | T1110.001 Brute Force: Password Guessing (Credential Access) | failed_login_count_exceeds:5<br>same_source_ip:true<br>time_window_minutes:10 | Medium |
| R008 | T1003.001 OS Credential Dumping: LSASS Memory (Credential Access) | process_access_target:lsass.exe<br>access_mask:0x1010<br>process_created:procdump.exe | High |

| Rule | Technique and tactic | IF conditions | Risk |
|---|---|---|---|
| R009 | T1552.001 Unsecured Credentials: Credentials In Files (Credential Access) | command_line_contains:findstr search_term_contains:password file_read_pattern:config | Medium |
| R010 | T1070.004 Indicator Removal: File Deletion (Defense Evasion) | deletion_command:del deleted_file_type:log file_deleted_in:System32 | Medium |
| R011 | T1027 Obfuscated Files or Information (Defense Evasion) | command_line_contains:FromBase64String command_line_entropy_exceeds:5.0 script_length_exceeds:1000 | Medium |
| R012 | T1562.001 Impair Defenses: Disable or Modify Tools (Defense Evasion) | service_stopped:WinDefend registry_key_modified:HKLM\SOFTWARE\Policies\Microsoft\Windows Defender command_line_contains:Set-MpPreference | High |
| R013 | T1218.011 System Binary Proxy Execution: Rundll32 (Defense Evasion) | process_created:rundll32.exe command_line_contains:javascript: parent_process_unusual:true | Medium |
| R014 | T1082 System Information Discovery (Discovery) | process_created:systeminfo.exe command_output_redirected:true | Low |
| R015 | T1083 File and Directory Discovery (Discovery) | command_line_contains:dir /s recursive_enumeration:true process_created:cmd.exe | Low |
| R016 | T1057 Process Discovery (Discovery) | process_created:tasklist.exe command_line_contains:/svc | Low |
| R017 | T1018 Remote System Discovery (Discovery) | process_created:net.exe command_line_contains:view /domain network_scan_detected:true | Low |
| R018 | T1566.001 Phishing: Spearphishing Attachment (Initial Access) | technique_detected:T1059.001 parent_process:winword.exe | High |

Table A.2 lists the mitigations attached to each rule. M9001 is team-defined and is not an official ATTACK mitigation.

**Table A.2 Mitigations attached to each rule**

| Rule | Technique | Mitigations |
|---|---|---|
| R001 | T1059.001 | M1042 Disable or Remove Feature or Program<br>M1026 Privileged Account Management<br>M1038 Execution Prevention |
| R002 | T1059.003 | M1038 Execution Prevention |
| R003 | T1053.005 | M1047 Audit<br>M1026 Privileged Account Management<br>M1018 User Account Management |
| R004 | T1547.001 | M1047 Audit |
| R005 | T1543.003 | M1018 User Account Management<br>M1022 Restrict File and Directory Permissions<br>M1024 Restrict Registry Permissions |
| R006 | T1136.001 | M1026 Privileged Account Management<br>M1032 Multi-factor Authentication<br>M1028 Operating System Configuration |
| R007 | T1110.001 | M1036 Account Use Policies<br>M1027 Password Policies<br>M1032 Multi-factor Authentication |
| R008 | T1003.001 | M1043 Credential Access Protection<br>M1025 Privileged Process Integrity<br>M1026 Privileged Account Management |
| R009 | T1552.001 | M1022 Restrict File and Directory Permissions<br>M1027 Password Policies<br>M1017 User Training |
| R010 | T1070.004 | M1029 Remote Data Storage |
| R011 | T1027 | M1049 Antivirus/Antimalware |
| R012 | T1562.001 | M1024 Restrict Registry Permissions<br>M1022 Restrict File and Directory Permissions<br>M1018 User Account Management |
| R013 | T1218.011 | M1038 Execution Prevention |
| R014 | T1082 | M9001 Detection-Only Guidance (team-defined) |
| R015 | T1083 | M9001 Detection-Only Guidance (team-defined) |
| R016 | T1057 | M9001 Detection-Only Guidance (team-defined) |
| R017 | T1018 | M1030 Network Segmentation<br>M9001 Detection-Only Guidance (team-defined) |
| R018 | T1566.001 | M1049 Antivirus/Antimalware<br>M1017 User Training |

## B. KEY TERMS

The table below is a quick reference for the terms used in this report.

**Table B.1 Key terms**

| Term | Meaning |
|---|---|
| Tactic | The attacker's goal at one stage of an attack, for example Credential Access. |
| Technique | The way a tactic is carried out, for example Brute Force (T1110). |
| Mitigation | A step a defender can take to stop or reduce a technique (ATTACK M-codes). |
| Fact | One observed behaviour written as attribute:value, for example failed_login_count_exceeds:5. |
| Rule | An IF–THEN statement linking facts (conditions) to a technique (conclusion). |
| Working memory | The place where known facts and new conclusions are stored. |
| Forward chaining | Reasoning that starts from the facts and keeps firing rules to reach conclusions. |
| Conflict set | All the rules that are currently eligible to fire. |
| Fixed point | The state where no more rules can fire, so reasoning stops. |
| Heuristic | A score used to guess which option is most promising. Here: matched ÷ total conditions. |
| Best-first search | Search that always explores the highest-scored option first, using a priority queue. |
| Partial match | A rule with only some of its conditions observed. Reported as a possible technique. |
| Goal-based agent | A program that observes, reasons and acts to reach a goal. |
| Explanation trace | The record of which rule fired, on which facts, and in what order. |

## C. ENVIRONMENT SETUP

The project is pure Python with no machine-learning dependencies. The setup used is Python 3.10 or newer (on Windows, "Add Python to PATH" is ticked during installation, and the installation is verified with python --version), VS Code with the Python and Jupyter extensions, and Git with a GitHub account for version control. A virtual environment was created inside the project folder so that package versions stay identical across machines. The third-party libraries were installed with the commands below.

```
python -m venv venv
pip install matplotlib pandas jupyter
pip install tabulate colorama flask
```

The modules json, heapq, dataclasses and collections are part of the Python standard library and need no installation.